# RTL Design Sherpa

**RTL AMBA Wishbone B4 Module Reference — Rev 0.90**

**July 2026**

# Table of Contents

# 1 Wishbone B4 Modules

**Location:** `rtl/amba/wb4/` **Test Location:** `val/amba/` **Status:** New (2026-09-09); master, slave, monitor, retry, clock-gated and CDC variants with sim and formal collateral. Family FULL sweep 2026-09-10 on one pinned seed base (`RDS_SEED_BASE=20260910`): 282 tests across the eight wb4 suites plus 8 for the AXI4-Lite bridge, all passing.

---

## 1.1 Overview

A Wishbone B4 master and slave pair (and a monitor for either), both in the **pipelined** mode the B4 revision added, behind the same FUB-side contract the APB4 pair uses: a command queue and a response queue, each a valid/ready handshake through a `gaxi_skid_buffer`. A FUB that already talks to `apb4_master` through `cmd_*`/`rsp_*` talks to `wb4_master` the same way; the differences are the field names and a 2-bit status in place of `pslverr`.

There is no Wishbone B5. B4 (2010) is the current revision of the specification and is what the `wb4` name records, the way `apb4` records AMBA 4 APB.

### 1.1.1 Protocol scope

| Signal set | Master (`m_wb_*`) | Slave (`s_wb_*`) |
|---|---|---|
| `CYC`, `STB`, `WE`, `ADR`, `DAT_W` (`DAT_O`), `SEL` | drives | receives |
| `STALL` | receives | drives (combinational from state, never from |

| Signal set | Master (m_wb_*) | Slave (s_wb_*) |
|---|---|---|
| | | STB) |
| ACK, ERR, RTY, DAT_R (DAT_I) | receives | drives (registered) |

- **Pipelined by default, classic by parameter.** With CLASSIC=0 a new STB every clock while STALL is low, and in-order termination. With CLASSIC=1 the blocks speak B4 standard ("classic") mode: the master holds the request on STB/CYC until the termination and ignores STALL; the slave never drives STALL, accepts a presentation once, and refuses an accept in the clock its termination is on the wire (the held STB is still there, and would otherwise be taken twice).
- **Match the mode to the peer.** The two modes do not mix, in either direction: a pipelined master drops STB after one clock, which a classic slave never accepts, and a classic master's held STB is accepted again every clock by a pipelined slave (B4 chapter 5). Both tests and both formal harnesses run each mode against a peer of the same mode.
- **RTY is a status, not a retry.** Every termination is returned to the FUB as rsp_status = ACK (0), ERR (1) or RTY (2). The master does not re-issue on RTY; the FUB decides, or wb4_retry decides for it. The encoding is wb4_pkg.
- **Burst hints are carried, not acted on.** CTI and BTE are advisory in B4. With USE_BURST_HINTS = 1 the master puts the FUB's hint on the wires with the transfer it belongs to and the slave hands the received hint to its FUB; neither changes behaviour, because deciding what a burst means is the peripheral's job. With the parameter at 0 the ports still exist and the bus reads CLASSIC/LINEAR, a legal non-burst cycle, which is what every consumer had before the hints landed.
- **Still not implemented, deliberately:** LOCK and the TGA/TGC/TGD tag signals.

### 1.1.2 Modules

- **wb4_master** - command/response queues in, pipelined Wishbone out; issue is credit-gated by the response queue so a termination can always be enqueued
- **wb4_slave** - pipelined Wishbone in, command/response queues out; in-order registered termination with an orphan-response guard
- **wb4_monitor** - watches either block's queues and reports completions, errors, timeouts, latency and address-range hits as monitor bus packets

tagged PROTOCOL_WB; in-order tracking queue, since B4 terminates in issue order

- **wb4_retry** - RTY retry on the FUB side of the master: in-order completion buffer, re-issue up to a budget with a delay, the FUB sees RTY only once the budget is spent; **wb4_master_retry** is the block and the master together
- **wb4_master_cg**, **wb4_slave_cg** - the clock-gated variants (amba_clock_gate_ctrl, runtime enable and idle count, output valids masked for the wake overlap)
- **wb4_slave_cdc** - the slave with its queues carried to another clock domain over two gaxi_fifo_async instances; **wb4_slave_cdc_cg** adds the gated bus clock
- **wb4_master_stub** / **wb4_slave_stub** - the pair with packed cmd/rsp vectors, for shim-style converters
- **wb4_pkg** - the response-status encoding shared by the family and the DV

### 1.1.3 Reset and clock naming

The ports are clk and aresetn (active-low), the repository convention, rather than Wishbone's CLK_I/RST_I (active-high). An integrator bridging to a Wishbone fabric that carries RST_I inverts it at the boundary; the reset macros inside the blocks are polarity-agnostic.

### 1.1.4 Test

Three tests in val/amba/, all through the RDS-DV framework's Wishbone B4 BFMs (CocoTBFramework.components.wb4: WB4Master, WB4Slave, WB4Monitor) and the GAXI BFMs on the FUB-side queues:

- test_wb4_master.py - the master alone: the framework slave answers on the bus from a memory model with ERR and RTY address windows, the monitor checks the wires, and every response on rsp_* is paired in order with the termination the slave produced.
- test_wb4_slave.py - the slave alone: the framework master drives the bus and the test is the FUB. Includes the abort case (CYC dropped with requests outstanding), which a master-slave loop can never produce, and proves the slave discards the late responses instead of pairing them with the next cycle.
- test_wb4_master_slave_loop.py - both back to back (rtl/amba/testcode/wb4_master_slave_loop.sv); the framework monitor and the wrapper's own protocol checks must agree.

Each asserts the peak in-flight count exceeds one, so the pipelined mode is proven exercised rather than assumed. Formal: `formal/amba/wb4_master/` and `formal/amba/wb4_slave/`.

# 2 wb4_master

## 2.1 Overview

`wb4_master` turns a command stream into Wishbone B4 **pipelined** bus cycles and returns every termination as a response. It is the Wishbone counterpart of `apb4_master`: the same `cmd_*`/`rsp_*` valid/ready queues on the FUB side, each a `gaxi_skid_buffer`. There is no state machine. `STB` follows the head of the command queue, the slave accepts a request on `STB && !STALL`, and every `ACK`, `ERR` or `RTY` is enqueued as it arrives. Terminations arrive in issue order (a B4 rule), so nothing is tagged.

**Protocol scope:** B4 pipelined. `RTY` is reported in `rsp_status`; the master never retries on its own. `CTI`/`BTE` burst hints are carried when `USE_BURST_HINTS = 1` and tied to CLASSIC/LINEAR otherwise; no `LOCK` or tag signals.

## 2.2 Parameters

| Parameter | Type | Default | Description |
|---|---|---|---|
| ADDR_WIDTH | int | 32 | Wishbone address width |
| DATA_WIDTH | int | 32 | Wishbone data width (port size) |
| CMD_DEPTH | int | 4 | Command queue depth in **entries**, 2..8 |
| RSP_DEPTH | int | 4 | Response queue depth in **entries**, 2..8; also the maximum transfers in flight |
| CLASSIC | int | 0 | 0 = B4 pipelined; 1 = B4 standard (“classic”) mode, see the family README. Match the peer: the modes do not mix |
| USE_BURST_HINTS | int | 0 | 0 = `CTI`/`BTE` are driven CLASSIC/LINEAR and `cmd_cti`/`cmd_bte` are |

| Parameter | Type | Default | Description |
|---|---|---|---|
| | | | ignored; 1 = the FUB's hints ride with their transfer onto the bus |
| SEL_WIDTH | int | DATA_WIDTH/8 | Byte-select width (derived) |

**RSP_DEPTH is the outstanding limit.** Wishbone gives a master no way to refuse a termination, so a request is only put on the bus when its response slot is already reserved. `RSP_DEPTH` therefore bounds both the queue and the pipeline depth; 4 sustains one transfer per clock against a slave with up to four clocks of termination latency.

## 2.3 Ports

```
module wb4_master
    import wb4_pkg::*;
#(
    parameter int ADDR_WIDTH = 32,
    parameter int DATA_WIDTH = 32,
    parameter int CMD_DEPTH  = 4,
    parameter int RSP_DEPTH  = 4,
    parameter int CLASSIC    = 0,
    parameter int SEL_WIDTH  = DATA_WIDTH / 8,
    // Short Parameters
    parameter int AW  = ADDR_WIDTH,
    parameter int DW  = DATA_WIDTH,
    parameter int SW  = SEL_WIDTH,
    parameter int STW = WB4_STATUS_WIDTH,
    parameter int CPW = 1 + AW + DW + SW,   // command packet: {we,
adr, dat, sel}
    parameter int RPW = STW + DW            // response packet:
{status, dat}
) (
    input  logic              clk,
    input  logic              aresetn,

    // Wishbone B4 pipelined master
    output logic              m_wb_CYC,
    output logic              m_wb_STB,
    output logic              m_wb_WE,
    output logic [AW-1:0]     m_wb_ADR,
    output logic [DW-1:0]     m_wb_DAT_W,
    output logic [SW-1:0]     m_wb_SEL,
    input  logic              m_wb_STALL,
    input  logic              m_wb_ACK,
    input  logic              m_wb_ERR,
```

```
    input  logic                m_wb_RTY,
    input  logic [DW-1:0]       m_wb_DAT_R,

    // Command queue (FUB -> bus)
    input  logic                cmd_valid,
    output logic                cmd_ready,
    input  logic                cmd_we,
    input  logic [AW-1:0]       cmd_adr,
    input  logic [DW-1:0]       cmd_dat,
    input  logic [SW-1:0]       cmd_sel,

    // Response queue (bus -> FUB)
    output logic                rsp_valid,
    input  logic                rsp_ready,
    output logic [STW-1:0]      rsp_status,
    output logic [DW-1:0]       rsp_dat
);
```

### 2.3.1 Clock and Reset

| Port | Width | Direction | Description |
|---|---|---|---|
| clk | 1 | Input | Bus clock |
| aresetn | 1 | Input | Active-low asynchronous reset (invert Wishbone `RST_I` at the boundary) |

### 2.3.2 Wishbone Master Interface

| Port | Width | Direction | Description |
|---|---|---|---|
| m_wb_CYC | 1 | Output | Cycle: high from the first `STB` until the last outstanding transfer terminates |
| m_wb_STB | 1 | Output | Strobe: a request is presented |
| m_wb_WE | 1 | Output | Write enable |
| m_wb_ADR | ADDR_WIDTH | Output | Address |
| m_wb_DAT_W | DATA_WIDTH | Output | Write data (`DAT_O` in the specification) |
| m_wb_SEL | SEL_WIDTH | Output | Byte select, reads and |

| Port | Width | Direction | Description |
|---|---|---|---|
| | | | writes |
| m_wb_STALL | 1 | Input | Slave cannot accept this clock; the request is held |
| m_wb_ACK | 1 | Input | Normal termination |
| m_wb_ERR | 1 | Input | Error termination |
| m_wb_RTY | 1 | Input | Retry termination |
| m_wb_DAT_R | DATA_WIDTH | Input | Read data (`DAT_I`) |

### 2.3.3 Command Interface

| Port | Width | Direction | Description |
|---|---|---|---|
| cmd_valid | 1 | Input | Command valid |
| cmd_ready | 1 | Output | Command queue has room |
| cmd_we | 1 | Input | Write (1) or read (0) |
| cmd_adr | ADDR_WIDTH | Input | Address |
| cmd_dat | DATA_WIDTH | Input | Write data |
| cmd_sel | SEL_WIDTH | Input | Byte select |

### 2.3.4 Response Interface

| Port | Width | Direction | Description |
|---|---|---|---|
| rsp_valid | 1 | Output | Response valid |
| rsp_ready | 1 | Input | FUB takes the response |
| rsp_status | 2 | Output | `WB4_RSP_ACK` (0), `WB4_RSP_ERR` (1), `WB4_RSP_RTY` (2) |
| rsp_dat | DATA_WIDTH | Output | Read data (undefined for writes, ERR and RTY) |

## 2.4 Functional Description

Two counters and three wires replace the FSM an APB master needs:

```
issue    = cmd head valid && r_reserved < RSP_DEPTH
STB      = issue
CYC      = issue || r_inflight != 0
accept   = STB && !STALL          -> pop command, r_inflight++,
r_reserved++
term     = CYC && (ACK|ERR|RTY)   -> push {status, DAT_R},
r_inflight--
rsp pop  = rsp_valid && rsp_ready -> r_reserved--
```

`r_inflight` counts transfers the slave has accepted but not terminated; `r_reserved` counts everything issued that the FUB has not yet taken off the response queue, which is exactly what can occupy that queue in the worst case. Reserving at issue rather than reading the queue's count back avoids the "count is stale by one" reasoning the APB master's back-to-back path needs.

**Classic mode** (`CLASSIC=1`): the command retires with its termination (`accept = term`), so nothing is in flight between clocks; `CYC` follows `STB`; `STALL` is ignored. The credit gate still guarantees the push at termination has room, since `r_reserved` then counts only queued responses.

`ERR` and `RTY` are mutually exclusive in the specification. If a slave asserts both, `ERR` wins, so a broken slave reads as an error rather than a retry.

### 2.4.1 Timing

| Path | Clocks |
|---|---|
| `cmd_valid` accepted to `STB` | 1 (command skid) |
| `ACK`/`ERR`/`RTY` to `rsp_valid` | 1 (response skid) |
| Sustained throughput | one request and one termination per clock |

A stalled request is held unchanged until accepted: the command skid's head does not move until `STB && !STALL`.

## 2.5 Usage Example

```
wb4_master #(
    .ADDR_WIDTH (32),
    .DATA_WIDTH (32),
    .CMD_DEPTH  (4),
    .RSP_DEPTH  (4)
) u_wb_master (
    .clk        (clk),
    .aresetn    (aresetn),
```

```
    .m_wb_CYC   (wb_cyc),   .m_wb_STB   (wb_stb),
    .m_wb_WE    (wb_we),    .m_wb_ADR   (wb_adr),
    .m_wb_DAT_W (wb_dat_o), .m_wb_SEL   (wb_sel),
    .m_wb_STALL (wb_stall), .m_wb_ACK   (wb_ack),
    .m_wb_ERR   (wb_err),   .m_wb_RTY   (wb_rty),
    .m_wb_DAT_R (wb_dat_i),
    .cmd_valid  (cmd_valid), .cmd_ready (cmd_ready),
    .cmd_we     (cmd_we),    .cmd_adr   (cmd_adr),
    .cmd_dat    (cmd_dat),   .cmd_sel   (cmd_sel),
    .rsp_valid  (rsp_valid), .rsp_ready (rsp_ready),
    .rsp_status (rsp_status),.rsp_dat   (rsp_dat)
);
```

## 2.6 Notes

- A termination that arrives with nothing in flight is a slave protocol violation. It is still enqueued (the FUB sees it) and reported in simulation.
- **Burst hints are carried, never acted on.** `CTI` and `BTE` are advisory in B4: the master puts the FUB's hint on the wires with the transfer it belongs to and changes nothing else. They ride inside the command queue, so a hint cannot slip onto a neighbouring transfer when the queue delays one. With `USE_BURST_HINTS = 0` the ports still exist and the bus reads CLASSIC/LINEAR, which is a legal non-burst Wishbone cycle.
- Under `ifdef FORMAL` the block asserts: `STB` implies `CYC`; `CYC` covers every in-flight transfer; a stalled request is held stable; the credit invariant `r_reserved <= RSP_DEPTH`; and that every termination found queue space.

## 2.7 Related

- wb4_slave - the mirror image
- apb4_master - the same FUB-side contract over APB
- gaxi_skid_buffer - both queues

## 2.8 Test

`val/amba/test_wb4_master.py` drives the block alone against the framework's Wishbone BFMs (`CocoTBFramework.components.wb4`), with the GAXI BFMs on the queues; `val/amba/test_wb4_master_slave_loop.py` runs it back to back with its counterpart. See the family README. Formal: `formal/amba/wb4_master/`.

# 3 wb4_master_cg

## 3.1 Overview

The **clock-gated variant** of wb4_master: an `amba_clock_gate_ctrl` instance produces a gated clock that feeds an otherwise unmodified `wb4_master`. Functionally identical to the base module; gating is enabled and tuned by input signals, not parameters, and `cfg_cg_enable = 0` gives the base behaviour exactly. See the Shared book's Clock-Gated Variants Guide and amba_clock_gate_ctrl for the architecture and the gate cell.

## 3.2 Parameters

In addition to all `wb4_master` parameters:

| Parameter | Type | Default | Description |
|---|---|---|---|
| CG_IDLE_COUNT_WIDTH | int | 4 | Width of the idle countdown; bounds the programmable idle threshold |

## 3.3 Ports

`wb4_master`'s ports plus:

| Port | Direction | Description |
|---|---|---|
| `cfg_cg_enable` | in | Global clock-gate enable |
| `cfg_cg_idle_count [ICW-1:0]` | in | Idle clocks before the clock is gated |
| `cg_gating` | out | Clock is gated now |
| `cg_idle` | out | Nothing pending (the controller's idle indicator) |

## 3.4 Functional Description

### 3.4.1 Wake-Up Terms

The registered wake term follows the family rule (peer VALIDs and every place work can be pending, never a peer READY):

| Term | Why |
|---|---|
| `cmd_valid` | a FUB command offered |
| response pending on the output | a termination waiting for the FUB |
| `m_wb_CYC` | a bus cycle open: requests queued, terminations due |

`rsp_ready` is deliberately absent: a FUB that parks its ready high while idle would otherwise hold the clock on forever.

### 3.4.2 Masks for the Wake-Latency Overlap

Gating can engage on the same edge pending work appears, and the wake takes a clock or two. The `rsp_valid` and the `cmd_ready` seen by the FUB are both held low while `cg_gating` is high, so a FUB never has a command taken, or a response retired, by a stopped clock. The masks only defer; once the pending term is high gating cannot engage, so a visible valid is never truncated. The bus side needs no mask: `CYC`/`STB` come from registered state that is idle whenever the clock may stop, and a slave terminates only inside a cycle.

## 3.5 Notes

- Formal: `formal/amba/wb4_master_cg/` proves the wrapper's glue contract with a clock-enable model of the gate cell (a derived clock is not provable in the repo's single-clock flow; the stopped clock itself is the cocotb test's job): reset state, no gating while a cycle is open on the bus or a response is visible, no gating with the enable low, `cmd_ready` low while gated, and a command offered wakes the clock by its third clock; covers gating, an ungated transfer, a response handed back and re-gating.
- The wake term is combinational into the controller (which registers it once), not a second local flop as in `apb4_master_cg`; see `formal/amba/apb4_slave_cg/KNOWN_BUG.md` for the latency that flop adds.

## 3.6 Related

- wb4_master, wb4_slave_cg
- apb4_master_cg - the same wrapper over APB

## 3.7 Test

`val/amba/test_wb4_master_cg.py` runs the `wb4_master` phases with gating enabled and checks, every clock, that the clock is never gated while a cycle is open, that the clock does gate after each phase drains, and that it never gates with the enable low.

# 4 wb4_master_retry

## 4.1 Overview

`wb4_master_retry` is wb4_retry in front of wb4_master: the master's command/response queue contract and Wishbone B4 ports, with RTY terminations retried up to `cfg_max_retries` times, `cfg_retry_delay` clocks apart, before the FUB sees one. Drop it in where `wb4_master` would go when the slave is known to retry.

## 4.2 Parameters

| Parameter | Type | Default | Description |
|---|---|---|---|
| ADDR_WIDTH | int | 32 | Wishbone address width |
| DATA_WIDTH | int | 32 | Wishbone data width |
| CMD_DEPTH | int | 4 | Master command queue depth |
| RSP_DEPTH | int | 4 | Master response queue depth; bounds the transfers on the bus |
| CLASSIC | int | 0 | 0 = B4 pipelined, 1 = B4 standard (classic) mode |
| INFLIGHT | int | 1 | Retry block completion-buffer depth. 1 = strict program order (see wb4_retry) |

With `INFLIGHT = 1` the bus carries one transfer at a time regardless of `RSP_DEPTH`; set `INFLIGHT` up to `RSP_DEPTH` for pipelined traffic that tolerates a retried transfer completing behind later ones.

## 4.3 Ports

`wb4_master`'s ports (`clk`, `aresetn`, `m_wb_*`, `cmd_*`, `rsp_*`) plus `cfg_max_retries [7:0]`, `cfg_retry_delay [15:0]`, `retry_count [31:0]` and `active_count [7:0]` from the retry block.

## 4.4 Usage Example

```
wb4_master_retry #(
    .ADDR_WIDTH (32),
    .DATA_WIDTH (32),
```

```
    .CMD_DEPTH  (4),
    .RSP_DEPTH  (4),
    .INFLIGHT   (1)
) u_wb_master (
    .clk (clk), .aresetn (rst_n),
    .cfg_max_retries (8'd3),
    .cfg_retry_delay (16'd16),
    .m_wb_CYC (wb_cyc), .m_wb_STB (wb_stb), .m_wb_WE
(wb_we), .m_wb_ADR (wb_adr),
    .m_wb_DAT_W (wb_dat_w), .m_wb_SEL (wb_sel),
    .m_wb_STALL (wb_stall), .m_wb_ACK (wb_ack), .m_wb_ERR
(wb_err), .m_wb_RTY (wb_rty),
    .m_wb_DAT_R (wb_dat_r),
    .cmd_valid (cmd_valid), .cmd_ready (cmd_ready), .cmd_we
(cmd_we), .cmd_adr (cmd_adr),
    .cmd_dat (cmd_dat), .cmd_sel (cmd_sel),
    .rsp_valid (rsp_valid), .rsp_ready (rsp_ready), .rsp_status
(rsp_status), .rsp_dat (rsp_dat),
    .retry_count (), .active_count ()
);
```

## 4.5 Related

- wb4_retry, wb4_master

## 4.6 Test

`val/amba/test_wb4_master_retry.py`; see wb4_retry.

# 5 wb4_master_stub

## 5.1 Overview

wb4_master with its FUB-side queues presented as a single **packed vector** instead of named fields, matching what `apb4_master_stub` and `apb5_master_stub` offer. A shim-style converter that already carries a packed command through its own pipeline connects one bus instead of six, and the packing lives here rather than being re-derived by every consumer.

Nothing else differs: the stub instantiates the ordinary block and only splices the vector.

## 5.2 Packing

Most significant field first. Both stubs use the same order, so a `wb4_master_stub` and a `wb4_slave_stub` connect directly:

```
cmd_data = {we, adr, dat, sel, cti, bte}      // CPW = 1 + AW + DW + SW
+ 3 + 2
rsp_data = {status, dat}                      // RPW = 2 + DW
```

`cti` and `bte` hold their bits whatever `USE_BURST_HINTS` is, so the vector width does not move with the parameter. With the hints off the inner block ignores them and the bus reads CLASSIC/LINEAR.

## 5.3 Parameters

Every parameter of wb4_master, plus the derived widths:

| Parameter | Value | Description |
|---|---|---|
| CPW | 1 + AW + DW + SW + 3 + 2 | Packed command width |
| RPW | 2 + DW | Packed response width |

## 5.4 Ports

wb4_master's Wishbone side unchanged. On the FUB side, the command arrives packed and the response leaves packed.

| Port | Direction | Description |
|---|---|---|
| `cmd_valid` / `cmd_ready` | in (cmd) / out (rsp) | Command handshake |
| `cmd_data [CPW-1:0]` | in (cmd) / out (rsp) | Packed command |
| `rsp_valid` / `rsp_ready` | in (cmd) / out (rsp) | Response handshake |
| `rsp_data [RPW-1:0]` | in (cmd) / out (rsp) | Packed response |

## 5.5 Related

- wb4_master - the block this wraps
- wb4_slave_stub - the other half of a stub pair
- wb4_pkg - the status encoding inside `rsp_data`

## 5.6 Test

`val/amba/test_wb4_stubs.py` checks the packing directly, in both directions and with the burst hints on and off. Two mutations fail it: swapping `dat`/`sel` in the master's unpack, and reversing the slave's response unpack.

# 6 wb4_monitor

## 6.1 Overview

wb4_monitor watches the cmd_*/rsp_* queues of a wb4_master or wb4_slave and reports what happens on the Wishbone side as monitor bus packets: a completion or an error per transfer, a timeout when a request or a termination is late, a latency figure when a transfer crosses a threshold, optional queue-activity debug events, and address-range violations through the shared apb_monitor_addr_check. It is the Wishbone member of the monitor family and has the same configuration, monitor bus and status ports as apb4_monitor; every packet is tagged PROTOCOL_WB (4'h5) and its event codes come from monitor_wb4_pkg.

The queues are the timing-convenient proxy for the bus, the same place apb4_monitor attaches: a command handshake is a request the master will put on the bus, a response handshake is the termination the slave returned. One difference from APB matters for the design: Wishbone B4 pipelined mode keeps several transfers open, and terminates them **in issue order**. The monitor therefore tracks transfers in an in-order queue, not a slot table. A slot table that pairs a response with “the first active slot” mis-pairs as soon as a freed slot is reused while an older one is still open, which pipelined traffic does constantly. The in-order queue pairs the response with the oldest open request by construction.

## 6.2 Parameters

| Parameter | Type | Default | Description |
|---|---|---|---|
| USE_MONITOR | bit | 1 | 0 removes the body and ties the outputs to idle |
| N_ADDR_RANGES | int | 0 | Address-range checker ranges; 0 leaves the checker out |
| ADDR_WIDTH | int | 32 | Wishbone address width |
| DATA_WIDTH | int | 32 | Wishbone data width |
| UNIT_ID | logic [7:0] | 8’h01 | Unit id stamped into every packet |
| AGENT_ID | logic [15:0] | 16’h000B | Agent id stamped into every packet |
| MAX_TRANSACTIONS | int | 8 | Depth of the in-order tracking queue; transfers open at once beyond it are reported, not tracked |
| MONITOR_FIFO_DEP | int | 8 | Event FIFO depth |

| Parameter | Type | Default | Description |
|---|---|---|---|
| TH | | | between event selection and the monitor bus |
| USE_BURST_HINTS | int | 0 | 1 reports the transfer's CTI in aux_data[7:5]; 0 leaves those bits zero and ignores cmd_cti |

MAX_TRANSACTIONS should match the RSP_DEPTH of the wb4_master (or the MAX_OUTSTANDING of the wb4_slave) it watches; those bound how many transfers can be open, so a queue of the same depth never overflows.

## 6.3 Ports

```
module wb4_monitor
    import monitor_common_pkg::*;   // PROTOCOL_WB, PktType*, packet
builder
    import monitor_wb4_pkg::*;      // WB_ERR_*, WB_TIMEOUT_*,
WB_COMPL_*, WB_PERF_*, WB_DEBUG_*
    import wb4_pkg::*;              // WB4_RSP_* status encoding
#(
    parameter bit USE_MONITOR         = 1'b1,
    parameter int N_ADDR_RANGES       = 0,
    parameter int ADDR_WIDTH          = 32,
    parameter int DATA_WIDTH          = 32,
    parameter logic [7:0]  UNIT_ID    = 8'h01,
    parameter logic [15:0] AGENT_ID   = 16'h000B,
    parameter int MAX_TRANSACTIONS    = 8,
    parameter int MONITOR_FIFO_DEPTH  = 8,
    parameter int USE_BURST_HINTS     = 0
)
(
    input  logic                      aclk,
    input  logic                      aresetn,

    // Command queue being watched
    input  logic                      cmd_valid,
    input  logic                      cmd_ready,
    input  logic                      cmd_we,
    input  logic [AW-1:0]             cmd_adr,
    input  logic [DW-1:0]             cmd_dat,
    input  logic [SW-1:0]             cmd_sel,
    input  logic [CTW-1:0]            cmd_cti,   // ignored when
USE_BURST_HINTS=0
```

```
    // Response queue being watched
    input  logic                        rsp_valid,
    input  logic                        rsp_ready,
    input  logic [1:0]                  rsp_status,     // WB4_RSP_ACK /
ERR / RTY
    input  logic [DW-1:0]               rsp_dat,

    // Configuration (same set as apb4_monitor)
    input  logic                        cfg_error_enable,
    input  logic                        cfg_timeout_enable,
    input  logic                        cfg_protocol_enable,
    input  logic                        cfg_slverr_enable,
    input  logic                        cfg_perf_enable,
    input  logic                        cfg_latency_enable,
    input  logic                        cfg_throughput_enable,
    input  logic                        cfg_debug_enable,
    input  logic                        cfg_trans_debug_enable,
    input  logic [3:0]                  cfg_debug_level,
    input  logic [15:0]                 cfg_cmd_timeout_cnt,
    input  logic [15:0]                 cfg_rsp_timeout_cnt,
    input  logic [31:0]                 cfg_latency_threshold,
    input  logic [15:0]                 cfg_throughput_threshold,

    // Address-range checker configuration (N_ADDR_RANGES > 0)
    input  logic                        cfg_addr_check_enable,
    input  logic [N-1:0]                cfg_addr_range_enable,
    input  logic [N-1:0][AW-1:0]        cfg_addr_range_low,
    input  logic [N-1:0][AW-1:0]        cfg_addr_range_high,

    input  monbus_timestamp_t           i_mon_time,

    // Monitor bus
    output logic                        monbus_valid,
    input  logic                        monbus_ready,
    output monitor_packet_t             monbus_packet,
    output monbus_timestamp_t           monbus_timestamp,

    // Status
    output logic [7:0]                  active_count,
    output logic [15:0]                 error_count,
    output logic [31:0]                 transaction_count
);
```

### 6.3.1 Command and Response Queues

The monitor is a pure observer: it samples `cmd_valid && cmd_ready` and `rsp_valid && rsp_ready` and never drives either queue. `cmd_dat` is accepted for port uniformity and not used; `rsp_dat` is reported only in the orphan-response packet.

### 6.3.2 Configuration

| Signal | Effect |
|---|---|
| `cfg_error_enable` | Master enable for error packets |
| `cfg_slverr_enable` | Report an `ERR` termination as `WB_ERR_ERR` (counted in `error_count` either way) |
| `cfg_protocol_enable` | Report a response with nothing outstanding as `WB_ERR_ORPHAN_RSP` |
| `cfg_timeout_enable` | Enable both timeouts; a count of 0 disables that timeout |
| `cfg_cmd_timeout_cnt` | Clocks a request may sit on `cmd_valid` without `cmd_ready` before `WB_TIMEOUT_CMD` |
| `cfg_rsp_timeout_cnt` | Clocks the oldest open transfer may wait for its termination before `WB_TIMEOUT_RSP` |
| `cfg_perf_enable`, `cfg_latency_enable` | Emit a latency packet when a transfer's latency exceeds `cfg_latency_threshold` |
| `cfg_debug_enable`, `cfg_trans_debug_enable` | Emit `WB_DEBUG_QUEUE_ACTIVE` / `WB_DEBUG_QUEUE_IDLE` on the tracking queue's edges |
| `cfg_throughput_*`, `cfg_debug_level` | Accepted for family uniformity; no packet today |

### 6.3.3 Status

`active_count` is the number of tracked transfers open right now, `transaction_count` the number terminated since reset, and `error_count` the number of `ERR` terminations (when `cfg_slverr_enable`), orphan responses and tracking overflows.

## 6.4 Functional Description

### 6.4.1 Transaction Tracking

A command handshake pushes `{we, adr, sel[3:0], cti, timestamp}` at the tail of the queue; a response handshake pops the head. The head is the transfer the response belongs to, because B4 terminates in issue order. Two edge cases are reported rather than guessed at:

- **Orphan response.** A response handshake with an empty queue. With `cfg_protocol_enable` it is a `WB_ERR_ORPHAN_RSP` packet carrying `rsp_dat` and the status; `transaction_count` does not move.
- **Tracking lost.** A command handshake with the queue full. The transfer is not tracked (its later response will show up as an orphan) and a `WB_ERR_TRACK_LOST` packet carries its address. Size `MAX_TRANSACTIONS` to the master's `RSP_DEPTH` and this never happens.

### 6.4.2 Event Detection

| Event | Packet type | Event code | event_data[31:0] | aux (event_data[39:32]) |
|---|---|---|---|---|
| Termination ACK | Completion | `WB_COMPL_READ` / `WB_COMPL_WRITE` | address | `{cti[2:0], sel[3:0], we}` |
| Termination RTY | Completion | `WB_COMPL_RTY` | address | `{cti[2:0], sel[3:0], we}` |
| Termination ERR | Error | `WB_ERR_ERR` | address | `{cti[2:0], sel[3:0], we}` |
| Response, nothing open | Error | `WB_ERR_ORPHAN_RSP` | `rsp_dat` | `{6'b0, status}` |
| Command, queue full | Error | `WB_ERR_TRACK_LOST` | address | `{cti[2:0], sel[3:0], we}` |
| `cmd_valid` stalled | Timeout | `WB_TIMEOUT_CMD` | address on `cmd_adr` | stall count (low 8 bits) |
| Head transfer late | Timeout | `WB_TIMEOUT_RSP` | head address | age (low 8 bits) |
| Latency over threshold | Perf | `WB_PERF_READ_LATENCY` / | latency in clocks | `{cti[2:0], sel[3:0],` |

| Event | Packet type | Event code | event_data[31:0] | aux (event_data[39:32]) |
|---|---|---|---|---|
| | | WB_PERF_WRITE_LATENCY | | we} |
| Queue non-empty / empty | Debug | WB_DEBUG_QUEUE_ACTIVE / WB_DEBUG_QUEUE_IDLE | occupancy | 0 |
| Address out of range | Error | WB_ERR_ADDR_RANGE | see apb_monitor_addr_check | see apb_monitor_addr_check |

cti[2:0] is zero on a USE_BURST_HINTS=0 build, which is what every consumer decoded before the hints existed, so the packet format did not change for anyone who does not ask for them. Only CTI is reported: the aux byte has exactly three spare bits, which is WB4_CTI_WIDTH, and BTE is deliberately left out rather than squeezed in. The hint travels **in the tracking entry**, so a completion reports the CTI of its own transfer even with several open at once – reading cmd_cti at completion time would report whatever request happened to be on the queue then.

RTY is a completion, not an error: the FUB decides whether to retry, and the monitor reports what the slave said. Each timeout fires **once**: the command timeout once per stall (it re-arms when the request is taken or withdrawn), the response timeout once per queue entry (a flag in the entry). When the head pops and the next entry is already older than the limit, that entry is reported on the following clock.

### 6.4.3 Monitor Packet Format

Standard 128-bit packet from create_monitor_packet with a 64-bit side-band timestamp taken from i_mon_time at emission:

- packet_type per the table above; protocol = PROTOCOL_WB (4'h5)
- event_code from monitor_wb4_pkg; channel_id = 0 (Wishbone has no IDs)
- unit_id = UNIT_ID, agent_id = AGENT_ID
- event_data[63:40] = 0, [39:32] = aux, [31:0] = value per the table

### 6.4.4 Event Priority and Loss

One event is written to the FIFO per clock, in the order error, timeout, perf, debug, completion; a lower-priority event that lands in the same clock as a higher one is dropped, and so is any event that arrives while the FIFO is full. The tracking queue does not wait for the packet, so active_count, transaction_count and error_count stay exact. This is the family's lossy-but-honest contract; size MONITOR_FIFO_DEPTH and the monitor bus consumer for the burstiest traffic the design can produce.

The event FIFO is a `gaxi_fifo_sync` in mux-read mode (`REGISTERED=0`): `rd_data` is valid in the clock of the read handshake, which is when the packet is built. The registered-read mode presents `rd_data` one clock after the handshake (the framework's `fifo_flop` BFM contract) and, read combinationally, re-presents the popped entry for a clock on back-to-back reads, so a burst of events would duplicate one packet and lose the next. This module's first test caught exactly that; see TASK-086 for the siblings.

## 6.5 Timing Characteristics

- Push and pop are registered from the queue handshakes; `active_count` updates on the clock after a handshake.
- A completion packet leaves the event FIFO one clock after the response handshake and the skid buffer one clock later, so `monbus_valid` for a termination rises two clocks after `rsp_valid && rsp_ready` when the bus is idle.
- `monbus_valid` is held until `monbus_ready` (skid buffer).

## 6.6 Usage Example

```
wb4_monitor #(
    .ADDR_WIDTH       (32),
    .DATA_WIDTH       (32),
    .UNIT_ID          (8'h02),
    .AGENT_ID         (16'h0020),
    .MAX_TRANSACTIONS (4),          // = wb4_master RSP_DEPTH
    .N_ADDR_RANGES    (2)
) u_wb_mon (
    .aclk (clk), .aresetn (rst_n),
    .cmd_valid (cmd_valid), .cmd_ready (cmd_ready), .cmd_we (cmd_we),
    .cmd_adr (cmd_adr), .cmd_dat (cmd_dat), .cmd_sel (cmd_sel),
    .rsp_valid (rsp_valid), .rsp_ready (rsp_ready),
    .rsp_status (rsp_status), .rsp_dat (rsp_dat),
    .cfg_error_enable (1'b1), .cfg_timeout_enable (1'b1),
    .cfg_protocol_enable (1'b1), .cfg_slverr_enable (1'b1),
    .cfg_perf_enable (1'b0), .cfg_latency_enable (1'b0),
    .cfg_throughput_enable (1'b0), .cfg_debug_enable (1'b0),
    .cfg_trans_debug_enable (1'b0), .cfg_debug_level (4'h0),
    .cfg_cmd_timeout_cnt (16'd256), .cfg_rsp_timeout_cnt (16'd1024),
    .cfg_latency_threshold (32'd0), .cfg_throughput_threshold (16'd0),
    .cfg_addr_check_enable (1'b1), .cfg_addr_range_enable (2'b11),
    .cfg_addr_range_low ('{32'h0000_0000, 32'h4000_0000}),
    .cfg_addr_range_high ('{32'h0FFF_FFFF, 32'h4FFF_FFFF}),
    .i_mon_time (mon_time),
    .monbus_valid (monbus_valid), .monbus_ready (monbus_ready),
    .monbus_packet (monbus_packet), .monbus_timestamp
(monbus_timestamp),
```

```
    .active_count (), .error_count (), .transaction_count ()
);
```

## 6.7 Notes

- Under `ifdef FORMAL` the block asserts the queue occupancy bound, that a pop only happens with something open, and that an orphan is only flagged when nothing is. `formal/amba/wb4_monitor/` adds the port-level properties (reset, protocol tag, valid-held, occupancy tracks the handshakes, `transaction_count` moves only on a pop) and covers every packet class.
- `USE_MONITOR=0` leaves the ports in place and ties `monbus_valid` and the counters to zero, so a build can drop the monitor without touching the wrapper.
- `monitor_wb4_pkg` is imported by this module only; it is deliberately not re-exported by `monitor_pkg`, so no existing consumer's filelist changes. `PROTOCOL_WB` itself lives in `monitor_common_pkg` (an additive enum entry).

## 6.8 Related

- wb4_master, wb4_slave - what it watches
- apb4_monitor - the family template
- apb_monitor_addr_check - the shared range checker, tagged `PROTOCOL_WB` here through its `PROTOCOL` parameter
- monitor_package_spec - packet format and protocol ids

## 6.9 Test

`val/amba/test_wb4_monitor.py` drives both queues with GAXI BFMs (a producer/consumer pair on each side, an age-based responder) and decodes the packets through the shared `MonbusSlave`/`TBClasses.monbus.parse` path. Phases: ACK/ERR/RTY mix in command order, several transfers open (in-order pairing), latency threshold, both timeouts once each, orphan, tracking overflow, debug edges, and address range on the `N_ADDR_RANGES=2` build. Four RTL mutations (timeout re-fire, swapped read/write codes, pairing off by one, orphan not reported) all fail the test.

# 7 wb4_pkg

The response-status encoding shared by every module in the Wishbone B4 family and by the Python bus functional models. It is one enum and one width constant, kept in a package so the value of ERR has exactly one definition.

## 7.1 Design Notes

**This is a package, not a module.** No ports, no clock. Compile order matters: it must be analysed before anything importing it, which is why `rtl/amba/filelists/wb4_pkg.f` exists and every consumer `-f` includes it rather than hand-listing the source.

**Why a status field at all.** Wishbone terminates a transfer with one of three wires, `ACK`, `ERR` or `RTY`. The FUB-side response queue carries the outcome as a 2-bit status instead, so a consumer reads one field rather than decoding three mutually exclusive strobes. `WB4_STATUS_WIDTH` sizes that field everywhere it appears.

**No test of its own, and none expected.** A package has no behaviour to simulate. It is verified by every module that imports it failing to elaborate if a declaration is wrong.

## 7.2 Declarations

```
package wb4_pkg;
    localparam int WB4_STATUS_WIDTH = 2;

    typedef enum logic [WB4_STATUS_WIDTH-1:0] {
        WB4_RSP_ACK = 2'd0,
        WB4_RSP_ERR = 2'd1,
        WB4_RSP_RTY = 2'd2
    } wb4_rsp_status_t;
endpackage
```

| Name | Value | Meaning |
|---|---|---|
| `WB4_STATUS_WIDTH` | 2 | Width of every `rsp_status` / `cmd`-side status field in the family |
| `WB4_RSP_ACK` | `2'd0` | Normal termination |
| `WB4_RSP_ERR` | `2'd1` | Slave signalled an error |
| `WB4_RSP_RTY` | `2'd2` | Slave asked for the transfer to be retried |
| *(unused)* | `2'd3` | Not an encoding; treated as reserved |

**RTY is a status, not an action.** `wb4_master` never re-issues on its own; it reports what the slave said and the FUB decides. Put `wb4_retry` in front of the master to absorb retries transparently up to a budget.

## 7.3 Burst hint encodings

Registered-feedback cycles (B4 chapter 4). Both are **advisory**: the family carries them when `USE_BURST_HINTS = 1` and neither the master nor the slave changes behaviour on them.

| wb4_cti_t | Value | Meaning |
|---|---|---|
| WB4_CTI_CLASSIC | 3'b000 | Classic cycle, no burst |
| WB4_CTI_CONST_ADDR | 3'b001 | Constant-address burst (a FIFO-like target) |
| WB4_CTI_INCR | 3'b010 | Incrementing burst |
| WB4_CTI_RSVD_3 .. WB4_CTI_RSVD_6 | 3'b011-3'b110 | Reserved by the spec |
| WB4_CTI_EOB | 3'b111 | End-of-burst: the last transfer of the run |

| wb4_bte_t | Value | Meaning |
|---|---|---|
| WB4_BTE_LINEAR | 2'b00 | Linear burst |
| WB4_BTE_WRAP4 | 2'b01 | 4-beat wrap |
| WB4_BTE_WRAP8 | 2'b10 | 8-beat wrap |
| WB4_BTE_WRAP16 | 2'b11 | 16-beat wrap |

`WB4_CTI_WIDTH` is 3 and `WB4_BTE_WIDTH` is 2. Both encodings put the non-burst case at zero on purpose, so a bus with the hint wires tied off is a legal classic bus rather than an illegal encoding.

## 7.4 Consumers

Every module in the family imports it: `wb4_master`, `wb4_slave`, `wb4_monitor`, `wb4_retry`, `wb4_master_retry`, the clock-gated and CDC variants, and the `axil4_to_wb4` bridge in `projects/components/converters`.

The DV side mirrors the same three values in `CocoTBFramework.components.shared.wb4_common` (`WB4_STATUS_ACK`, `WB4_STATUS_ERR`, `WB4_STATUS_RTY`), and `monitor_wb4_pkg` carries the separate event codes the monitor emits.

## 7.5 Related

- Wishbone B4 family README - the protocol scope and the mode rules
- `monitor_wb4_pkg` - the monitor's event codes, a different package

# 8 wb4_retry

## 8.1 Overview

`wb4_retry` sits on the FUB side of `wb4_master` and turns a Wishbone RTY into a retry. Every command the FUB issues is held in an in-order completion buffer until its answer has been handed back. ACK and ERR complete the entry as they arrive. RTY puts the entry back on the issue path after `cfg_retry_delay` clocks, until `cfg_max_retries` re-issues have been spent; only then does the FUB see the RTY. Responses reach the FUB in command order whatever happened on the bus, and re-issues always take priority over new commands.

`cfg_max_retries = 0` makes the block a pass-through. `wb4_master_retry` is the block and the master together, with the master's ports.

**Order on the bus.** A retried command re-appears on the bus behind the commands issued after it, which with `INFLIGHT > 1` may already have terminated. The FUB still gets its responses in order, but a read issued behind a write that was retried can observe memory from before that write. `INFLIGHT = 1`, the default, keeps program order exactly: one command on the bus at a time, retries included. Use `INFLIGHT > 1` only for traffic with no ordering dependence across transfers.

## 8.2 Parameters

| Parameter | Type | Default | Description |
|---|---|---|---|
| ADDR_WIDTH | int | 32 | Wishbone address width |
| DATA_WIDTH | int | 32 | Wishbone data width |
| INFLIGHT | int | 1 | Completion-buffer entries: commands accepted from the FUB and not yet answered. 1 = strict program order |

## 8.3 Ports

```
module wb4_retry
    import wb4_pkg::*;
#(
    parameter int ADDR_WIDTH = 32,
    parameter int DATA_WIDTH = 32,
    parameter int INFLIGHT   = 1
)
(
    input  logic              clk,
    input  logic              aresetn,
```

```
    input  logic [7:0]        cfg_max_retries,   // 0 = pass RTY
through
    input  logic [15:0]       cfg_retry_delay,   // clocks from RTY to
re-issue

    // FUB side: the same contract wb4_master offers
    input  logic              cmd_valid,
    output logic              cmd_ready,
    input  logic              cmd_we,
    input  logic [AW-1:0]     cmd_adr,
    input  logic [DW-1:0]     cmd_dat,
    input  logic [SW-1:0]     cmd_sel,
    output logic              rsp_valid,
    input  logic              rsp_ready,
    output logic [1:0]        rsp_status,        // WB4_RSP_ACK /
ERR / RTY
    output logic [DW-1:0]     rsp_dat,

    // wb4_master side
    output logic              mst_cmd_valid,
    input  logic              mst_cmd_ready,
    output logic              mst_cmd_we,
    output logic [AW-1:0]     mst_cmd_adr,
    output logic [DW-1:0]     mst_cmd_dat,
    output logic [SW-1:0]     mst_cmd_sel,
    input  logic              mst_rsp_valid,
    output logic              mst_rsp_ready,
    input  logic [1:0]        mst_rsp_status,
    input  logic [DW-1:0]     mst_rsp_dat,

    output logic [31:0]       retry_count,       // re-issues since
reset
    output logic [7:0]        active_count       // entries in the
buffer
);
```

### 8.3.1 Configuration

`cfg_max_retries` is the number of re-issues allowed per command, not the number of attempts: with 3, a command is presented to the bus at most four times. `cfg_retry_delay` is the number of clocks between the RTY and the re-issue; 0 re-issues on the next clock the master can take a command. Both are sampled when used, so they can change between transfers.

### 8.3.2 Status

`retry_count` counts every re-issue since reset. `active_count` is the number of commands accepted from the FUB and not yet answered, at most `INFLIGHT`.

## 8.4 Functional Description

Each entry of the completion buffer holds the command (`we`, `adr`, `dat`, `sel`), a retry count, a delay timer and, once answered, the status and read data. Entries are allocated at the tail when a command is accepted and released at the head when the FUB takes the response, so the FUB order is the buffer order. A separate issue log records which entry each command handed to the master belongs to, in the order the master received them; Wishbone terminates in that order, so the log head is always the entry a termination belongs to.

| Event | Effect |
|---|---|
| `cmd_valid && cmd_ready` | Entry allocated at the tail and forwarded to `mst_cmd_*` in the same clock |
| `mst_rsp_*` handshake, `ACK` or `ERR` | Log-head entry marked done with the status and data |
| `mst_rsp_*` handshake, `RTY`, retries left | Entry marked waiting, retry count +1, timer loaded with `cfg_retry_delay`, `retry_count` +1 |
| `mst_rsp_*` handshake, `RTY`, budget spent | Entry marked done with `RTY` |
| Waiting entry with timer at 0 | Re-issued on `mst_cmd_*` before any new command (oldest first) |
| Head entry done | `rsp_valid`; released on the handshake |

`cmd_ready` is low while a retry is pending issue, while the buffer is full, and while the master cannot take a command, so a FUB never has a command accepted that would be reordered behind a retry. `mst_rsp_ready` is high whenever a command is on the bus: the termination's entry is waiting for it, so the master is never back-pressured on a response.

## 8.5 Timing

- A new command reaches `mst_cmd_*` combinationally in the clock it is accepted; the master's own skid buffer registers it.
- A termination is recorded in the clock it arrives; `rsp_valid` for the head follows one clock later (registered state).

- With `INFLIGHT = 1` the bus carries one transfer at a time, so the sustained rate is one transfer per round trip. With `INFLIGHT = N` and a master `RSP_DEPTH >= N`, up to N transfers are open.

## 8.6 Notes

- Under `ifdef FORMAL` the block asserts the occupancy bounds of both the buffer and the issue log, that everything on the bus is an allocated entry, that a retry is never issued in the clock a new command is accepted, and that the FUB only ever sees the head once it is done. `formal/amba/wb4_retry/` adds the port-level contract (payload equality of every re-issue, `1 + retries` issues per command, the budget and the delay, `RTY` to the FUB only once the budget is spent) at `INFLIGHT = 1` and the bounds at `INFLIGHT = 2`.
- The block does not know why the slave retried. Back-off is a fixed delay; a slave that needs a longer or growing gap needs a larger `cfg_retry_delay` from the controlling software.

## 8.7 Related

- wb4_master_retry - this block with `wb4_master` behind it
- wb4_master - what it drives
- axil4_to_wb4 - a bridge that maps `RTY` to an AXI error; put this block between it and the master to retry instead

## 8.8 Test

`val/amba/test_wb4_master_retry.py`, through the wrapper: the framework Wishbone slave answers with address windows that retry a bounded number of times, retry forever, or error, so every FUB response, the number of re-issues (`retry_count`, the monitor's transfer count, the slave's `RTY` count) and the read data follow from the model. Formal: `formal/amba/wb4_retry/`.

# 9 wb4_slave

## 9.1 Overview

`wb4_slave` accepts Wishbone B4 **pipelined** transfers one per clock and presents each on a command queue; it terminates them, in order, from a response queue with a registered `ACK`, `ERR` or `RTY` and read data. It is the Wishbone counterpart of `apb4_slave`, with the same `cmd_*`/`rsp_*`

valid/ready contract on the FUB side. There is no state machine: `STALL` is the inverse of "can accept", and the termination side is one counter and one register.

**Protocol scope:** B4 pipelined. The FUB chooses the termination through `rsp_status`, so a FUB can answer RTY without the slave knowing why.

## 9.2 Parameters

| Parameter | Type | Default | Description |
|---|---|---|---|
| ADDR_WIDTH | int | 32 | Wishbone address width |
| DATA_WIDTH | int | 32 | Wishbone data width (port size) |
| CMD_DEPTH | int | 2 | Command queue depth in **entries**, 2..8 |
| RSP_DEPTH | int | 2 | Response queue depth in **entries**, 2..8 |
| MAX_OUTSTANDING | int | 16 | Transfers accepted but not yet terminated before `STALL` asserts |
| CLASSIC | int | 0 | 0 = B4 pipelined; 1 = B4 standard ("classic") mode, see the family README. Match the peer: the modes do not mix |
| USE_BURST_HINTS | int | 0 | 0 = the bus `CTI`/`BTE` inputs are ignored and `cmd_cti`/`cmd_bte` read CLASSIC/LINEAR; 1 = each transfer's hints are handed to the FUB with it |
| SEL_WIDTH | int | DATA_WIDTH/8 | Byte-select width (derived) |

`MAX_OUTSTANDING` bounds the FUB's in-order pipeline, not the master's: a master's own limit is its response queue. Set it to at least the number of commands the FUB can hold before it produces the first response, or the bus stalls short of the FUB's throughput.

## 9.3 Ports

```
module wb4_slave
    import wb4_pkg::*;
#(
    parameter int ADDR_WIDTH      = 32,
    parameter int DATA_WIDTH      = 32,
    parameter int CMD_DEPTH       = 2,
    parameter int RSP_DEPTH       = 2,
    parameter int MAX_OUTSTANDING = 16,
    parameter int CLASSIC         = 0,
    parameter int SEL_WIDTH       = DATA_WIDTH / 8,
    // Short Parameters
    parameter int AW  = ADDR_WIDTH,
    parameter int DW  = DATA_WIDTH,
    parameter int SW  = SEL_WIDTH,
    parameter int STW = WB4_STATUS_WIDTH,
    parameter int CPW = 1 + AW + DW + SW,   // command packet: {we,
adr, dat, sel}
    parameter int RPW = STW + DW            // response packet:
{status, dat}
) (
    input  logic             clk,
    input  logic             aresetn,

    // Wishbone B4 pipelined slave
    input  logic             s_wb_CYC,
    input  logic             s_wb_STB,
    input  logic             s_wb_WE,
    input  logic [AW-1:0]    s_wb_ADR,
    input  logic [DW-1:0]    s_wb_DAT_W,
    input  logic [SW-1:0]    s_wb_SEL,
    output logic             s_wb_STALL,
    output logic             s_wb_ACK,
    output logic             s_wb_ERR,
    output logic             s_wb_RTY,
    output logic [DW-1:0]    s_wb_DAT_R,

    // Command queue (bus -> FUB)
    output logic             cmd_valid,
    input  logic             cmd_ready,
    output logic             cmd_we,
    output logic [AW-1:0]    cmd_adr,
    output logic [DW-1:0]    cmd_dat,
    output logic [SW-1:0]    cmd_sel,

    // Response queue (FUB -> bus)
    input  logic             rsp_valid,
```

```
    output logic                rsp_ready,
    input  logic [STW-1:0]      rsp_status,
    input  logic [DW-1:0]       rsp_dat
);
```

### 9.3.1 Clock and Reset

| Port | Width | Direction | Description |
|---|---|---|---|
| clk | 1 | Input | Bus clock |
| aresetn | 1 | Input | Active-low asynchronous reset |

### 9.3.2 Wishbone Slave Interface

| Port | Width | Direction | Description |
|---|---|---|---|
| s_wb_CYC | 1 | Input | Cycle |
| s_wb_STB | 1 | Input | Strobe: a request is presented |
| s_wb_WE | 1 | Input | Write enable |
| s_wb_ADR | ADDR_WIDTH | Input | Address |
| s_wb_DAT_W | DATA_WIDTH | Input | Write data (`DAT_I` at the slave) |
| s_wb_SEL | SEL_WIDTH | Input | Byte select |
| s_wb_STALL | 1 | Output | Cannot accept this clock; combinational from queue room and the outstanding count, never from `STB` |
| s_wb_ACK | 1 | Output | Normal termination (registered, one clock) |
| s_wb_ERR | 1 | Output | Error termination (registered, one clock) |
| s_wb_RTY | 1 | Output | Retry termination (registered, one clock) |
| s_wb_DAT_R | DATA_WIDTH | Output | Read data (`DAT_O` at the slave); holds between terminations |

### 9.3.3 Command Interface

| Port | Width | Direction | Description |
|---|---|---|---|
| cmd_valid | 1 | Output | Command valid |
| cmd_ready | 1 | Input | FUB takes the command |
| cmd_we | 1 | Output | Write (1) or read (0) |
| cmd_adr | ADDR_WIDTH | Output | Address |
| cmd_dat | DATA_WIDTH | Output | Write data |
| cmd_sel | SEL_WIDTH | Output | Byte select |

### 9.3.4 Response Interface

| Port | Width | Direction | Description |
|---|---|---|---|
| rsp_valid | 1 | Input | Response valid |
| rsp_ready | 1 | Output | Response queue has room |
| rsp_status | 2 | Input | `WB4_RSP_ACK` (0), `WB4_RSP_ERR` (1), `WB4_RSP_RTY` (2) |
| rsp_dat | DATA_WIDTH | Input | Read data |

## 9.4 Functional Description

```
STALL  = !cmd queue room || r_outstanding == MAX_OUTSTANDING
accept = CYC && STB && !STALL     -> push {we, adr, dat, sel},
r_outstanding++
term   = rsp head valid && r_outstanding != 0 && CYC
         -> register ACK|ERR|RTY from status, DAT_R, pop,
r_outstanding--
orphan = rsp head valid && (r_outstanding == 0 || r_abandoned != 0) ->
pop and drop
abort  = !CYC && r_outstanding != 0 -> r_abandoned += r_outstanding,
r_outstanding <= 0
```

**Classic mode** (`CLASSIC=1`): `STALL` is driven low; a request is accepted when nothing is outstanding and no termination is on the wire this clock, so a held presentation is taken exactly once; `MAX_OUTSTANDING` is effectively 1.

**Orphan guard.** A response with nothing outstanding cannot belong to any transfer (a duplicate from the FUB, or a response to a transfer the master abandoned). It is dropped, and reported in simulation. Left in the queue it would terminate the *next* transfer and every later response would be off by one: the positional mis-pairing `apb4_slave`'s guard exists for.

**Abort.** A master that drops `CYC` with transfers outstanding has ended the cycle. The outstanding count moves to an *abandoned* count, and that many later responses from the FUB are dropped as they arrive, even if the master has started a new cycle by then. Without that, a late response for an abandoned transfer would terminate the new cycle's first transfer, which is exactly what the slave test's abort phase caught before the count existed. Terminations are only ever driven inside a cycle, and never while a response is still owed to an abandoned transfer.

### 9.4.1 Timing

| Path | Clocks |
|---|---|
| `STB` accepted to `cmd_valid` | 1 (command skid) |
| `rsp_valid` to `ACK`/`ERR`/`RTY` | 2 (response skid, then the output register) |
| Sustained throughput | one accept and one termination per clock |

## 9.5 Usage Example

```
wb4_slave #(
    .ADDR_WIDTH      (32),
    .DATA_WIDTH      (32),
    .CMD_DEPTH       (2),
    .RSP_DEPTH       (2),
    .MAX_OUTSTANDING (16)
) u_wb_slave (
    .clk        (clk),
    .aresetn    (aresetn),
    .s_wb_CYC   (wb_cyc),   .s_wb_STB   (wb_stb),
    .s_wb_WE    (wb_we),    .s_wb_ADR   (wb_adr),
    .s_wb_DAT_W (wb_dat_i), .s_wb_SEL   (wb_sel),
    .s_wb_STALL (wb_stall), .s_wb_ACK   (wb_ack),
    .s_wb_ERR   (wb_err),   .s_wb_RTY   (wb_rty),
    .s_wb_DAT_R (wb_dat_o),
    .cmd_valid  (cmd_valid), .cmd_ready (cmd_ready),
    .cmd_we     (cmd_we),    .cmd_adr   (cmd_adr),
    .cmd_dat    (cmd_dat),   .cmd_sel   (cmd_sel),
    .rsp_valid  (rsp_valid), .rsp_ready (rsp_ready),
    .rsp_status (rsp_status),.rsp_dat   (rsp_dat)
);
```

## 9.6 Notes

- The FUB must answer commands in the order it received them; the slave pairs responses to transfers by position.
- Under `ifdef FORMAL` the block asserts: at most one of ACK/ERR/RTY per clock; a termination only for a transfer that was outstanding inside a cycle; `r_outstanding <= MAX_OUTSTANDING`; and that every accept found queue room.

## 9.7 Related

- wb4_master - the mirror image
- apb4_slave - the same FUB-side contract over APB
- gaxi_skid_buffer - both queues

## 9.8 Test

`val/amba/test_wb4_slave.py` drives the block alone against the framework's Wishbone BFMs (`CocoTBFramework.components.wb4`), with the GAXI BFMs on the queues; `val/amba/test_wb4_master_slave_loop.py` runs it back to back with its counterpart. See the family README. Formal: `formal/amba/wb4_slave/`.

# 10 wb4_slave_cdc

## 10.1 Overview

wb4_slave with its command and response queues carried across a clock-domain boundary. The Wishbone bus lives in `wb_clk`; the FUB's `cmd_*`/`rsp_*` queues live in `aclk`. Two `gaxi_fifo_async` instances do the crossing (commands `wb_clk -> aclk`, responses `aclk -> wb_clk`), the same structure as apb4_slave_cdc. In-order termination is preserved because both crossings are FIFOs.

## 10.2 Parameters

In addition to all `wb4_slave` parameters:

| Parameter | Type | Default | Description |
|---|---|---|---|
| CDC_DEPTH | int | 4 | Async FIFO depth, floored at 4; power of two preferred (Gray |

| Parameter | Type | Default | Description |
|---|---|---|---|
| | | | pointers) |
| USE_JOHNSON | int | 0 | 0 = Gray pointers (power-of-two depth), 1 = Johnson pointers (any depth) |

## 10.3 Ports

`wb4_slave`'s ports, with the clock and reset split by domain:

| Port | Domain | Description |
|---|---|---|
| `wb_clk`, `wb_resetn` | Wishbone | the `s_wb_*` bus |
| `aclk`, `aresetn` | FUB | the `cmd_*` / `rsp_*` queues |

## 10.4 Functional Description

### 10.4.1 Clock Domains

The slave itself, its skid buffers, the `STALL` logic and the registered terminations are all in `wb_clk`. `MAX_OUTSTANDING` bounds the requests accepted while responses are pending, so the response FIFO can never overflow the slave's response queue whatever the clock ratio.

### 10.4.2 CDC Structure

```
 s_wb_* (wb_clk) --> wb4_slave --> cmd FIFO (wb_clk -> aclk) --> cmd_*
(aclk)
                                  <-- rsp FIFO (aclk -> wb_clk) <-- rsp_*
(aclk)
```

Each FIFO is a `gaxi_fifo_async` with two-flop pointer synchronisers (`N_FLOP_CROSS = 2`). Throughput is one transfer per clock of the slower side once the pointers have crossed; latency is a few clocks of each domain per direction.

### 10.4.3 Reset Behavior

Each FIFO resets its own side's pointers from that side's reset. While a reset is asserted the resetting side is self-consistent, but a **one-sided reset with transfers in the FIFOs is not safe**: a write-side reset alone lets the read side see phantom occupancy (it fabricates entries), and a read-side reset alone replays consumed entries. Quiesce the bus before resetting one side. The full analysis is in apb4_slave_cdc, Reset Behavior, and applies here unchanged.

## 10.5 Related

- wb4_slave, wb4_slave_cg
- gaxi_fifo_async - the crossing

## 10.6 Test

`val/amba/test_wb4_slave_cdc.py` runs the `wb4_slave` phases with the Wishbone BFMs on `wb_clk` and the queue BFMs on `aclk` at several clock ratios in both directions (bus faster, bus slower). Formal: `formal/amba/wb4_slave_cdc/`, single-clock model (both domains on one clock) for the port-level contract, as the APB CDC harness does.

# 11 wb4_slave_cdc_cg

## 11.1 Overview

wb4_slave_cdc with the **Wishbone-side clock gated**, the combined variant matching `apb4_slave_cdc_cg` / `apb5_slave_cdc_cg`.

Only the Wishbone domain is gated. The FUB domain keeps running, so a consumer can go on draining commands and posting responses while the bus sleeps; the async FIFOs make that safe. Gating both would need a second controller and buys nothing a caller cannot do by gating its own clock.

## 11.2 Parameters

Every `wb4_slave_cdc` parameter, plus `CG_IDLE_COUNT_WIDTH` (default 4).

## 11.3 Ports

`wb4_slave_cdc`'s ports plus `cfg_cg_enable`, `cfg_cg_idle_count`, `cg_gating` and `cg_idle`, all in the `wb_clk` domain.

## 11.4 The wake term has to be single-domain

This is the one thing worth reading before changing it. The wake term is `wb4_slave_cdc`'s `wb_busy` output, which is built **only** from `wb_clk` signals: a cycle open on the bus, a command waiting to cross, or a response that has crossed and not yet been driven.

Nothing from the `aclk` side may be used, because sampling it in `wb_clk` would be an unsynchronised crossing. That is the mistake this wrapper invites and the reason `wb_busy` exists rather than the wake term being assembled here from whatever looked convenient.

For the same reason `cmd_valid` is **not** masked with `cg_gating` the way wb4_slave_cg masks it: `cmd_valid` lives in `aclk`, and masking it with a `wb_clk`-domain signal would be exactly that crossing. `s_wb_STALL` **is** held high while gated, for the reason `wb4_slave_cg` gives: a pipelined accept is `STB && !STALL` in the master's clock, and a frozen "room" would let a master move on from a request the slave never sampled.

## 11.5 Reset

`wb4_slave_cdc`'s one-sided reset caveat applies unchanged: quiesce the bus before resetting one side.

## 11.6 Test

`val/amba/test_wb4_slave_cdc.py` carries a `cg` dimension that selects this module in place of `wb4_slave_cdc` and drives the gating configuration; the same traffic, abort and post-abort phases run at several clock ratios in both directions.

## 11.7 Related

- wb4_slave_cdc, wb4_slave_cg

# 12 wb4_slave_cg

## 12.1 Overview

The **clock-gated variant** of wb4_slave: an `amba_clock_gate_ctrl` instance produces a gated clock that feeds an otherwise unmodified `wb4_slave`. Functionally identical to the base module; `cfg_cg_enable = 0` gives the base behaviour exactly. See the Shared book's Clock-Gated Variants Guide and amba_clock_gate_ctrl.

## 12.2 Parameters

In addition to all `wb4_slave` parameters:

| Parameter | Type | Default | Description |
|---|---|---|---|
| CG_IDLE_COUNT_WIDTH | int | 4 | Width of the idle countdown; bounds the programmable idle threshold |

## 12.3 Ports

`wb4_slave`'s ports plus `cfg_cg_enable`, `cfg_cg_idle_count`, `cg_gating` and `cg_idle`, as on wb4_master_cg.

## 12.4 Functional Description

### 12.4.1 Wake-Up Terms

| Term | Why |
|---|---|
| `s_wb_CYC` | a bus cycle open: requests arriving, terminations due |
| `rsp_valid` | a FUB response offered |
| command pending on the output | a request waiting for the FUB |

`cmd_ready` is deliberately absent.

### 12.4.2 Masks for the Wake-Latency Overlap

A stopped clock cannot register an accept, and the wake takes a clock or two after the activity appears. Three signals are therefore masked while `cg_gating` is high:

| Signal | Masked to | Why |
|---|---|---|
| `cmd_valid` (to the FUB) | low | a consumer must not retire a command the slave cannot see leave |
| `rsp_ready` (to the FUB) | low | a FUB must not have a response taken by a stopped clock |
| `s_wb_STALL` (to the master) | **high** | in pipelined mode an accept is `STB && !STALL` in the master's clock; a frozen "room" would let the master move on from a request the slave never sampled |

`STALL` stays high until the clock runs, and the master holds its request while stalled (a B4 rule), so nothing is lost and nothing is duplicated. Classic mode holds the request until termination and needs no `STALL`, but is masked the same way.

## 12.5 Notes

- Formal: `formal/amba/wb4_slave_cg/` proves the wrapper's glue contract with a clock-enable model of the gate cell (a derived clock is not provable in the repo's single-clock flow; the stopped clock itself is the cocotb test's job): reset state, gating clears by the third clock of an open cycle, no gating while a command is pending or with the enable low, no termination and no FUB handshake while gated, `STALL` high while gated; covers gating, an ungated accept, a termination and re-gating.
- The wake term is combinational into the controller (which registers it once), not a second local flop as in `apb4_slave_cg`; see `formal/amba/apb4_slave_cg/KNOWN_BUG.md` for the latency that flop adds.

## 12.6 Related

- wb4_slave, wb4_master_cg
- apb4_slave_cg - the same wrapper over APB

## 12.7 Test

`val/amba/test_wb4_slave_cg.py` runs the `wb4_slave` phases with gating enabled and the same three checks as the master's test.

# 13 wb4_slave_stub

## 13.1 Overview

wb4_slave with its FUB-side queues presented as a single **packed vector** instead of named fields, matching what `apb4_slave_stub` and `apb5_slave_stub` offer. A shim-style converter that already carries a packed command through its own pipeline connects one bus instead of six, and the packing lives here rather than being re-derived by every consumer.

Nothing else differs: the stub instantiates the ordinary block and only splices the vector.

## 13.2 Packing

Most significant field first. Both stubs use the same order, so a `wb4_master_stub` and a `wb4_slave_stub` connect directly:

```
cmd_data = {we, adr, dat, sel, cti, bte}      // CPW = 1 + AW + DW + SW
+ 3 + 2
rsp_data = {status, dat}                      // RPW = 2 + DW
```

`cti` and `bte` hold their bits whatever `USE_BURST_HINTS` is, so the vector width does not move with the parameter. With the hints off the inner block ignores them and the bus reads CLASSIC/LINEAR.

## 13.3 Parameters

Every parameter of wb4_slave, plus the derived widths:

| Parameter | Value | Description |
|---|---|---|
| CPW | 1 + AW + DW + SW + 3 + 2 | Packed command width |
| RPW | 2 + DW | Packed response width |

## 13.4 Ports

wb4_slave's Wishbone side unchanged. On the FUB side, the command leaves packed and the response arrives packed.

| Port | Direction | Description |
|---|---|---|
| `cmd_valid` / `cmd_ready` | out (cmd) / in (rsp) | Command handshake |
| `cmd_data [CPW-1:0]` | out (cmd) / in (rsp) | Packed command |
| `rsp_valid` / `rsp_ready` | out (cmd) / in (rsp) | Response handshake |
| `rsp_data [RPW-1:0]` | out (cmd) / in (rsp) | Packed response |

## 13.5 Related

- wb4_slave - the block this wraps
- wb4_master_stub - the other half of a stub pair
- wb4_pkg - the status encoding inside `rsp_data`

## 13.6 Test

`val/amba/test_wb4_stubs.py` checks the packing directly, in both directions and with the burst hints on and off. Two mutations fail it: swapping `dat`/`sel` in the master's unpack, and reversing the slave's response unpack.